BROKEN OBJECT LEVEL AUTHORIZATION (IDOR) BANKING DEMO

SUBMITTED BY

ANSHIKA RAJ (23BCI0100)
ADITYA PRAKASH (23BCI0089)
ARYAN SHARMA (23BCI0154)

SUBMITTING TO: HELENSHARMILA A

In partial fulfillment of the requirements for the completion of the course

BCSE302L WEB APPLICATION SECURITY



SCHOOL OF COMPUTER SCIENCE AND ENGINEERING
VELLORE INSTITUTE OF TECHNOLOGY, VELLORE

AUGUST 2026

TABLE OF CONTENTS

Sl. No.	Contents	Page No.
1.	Introduction	3
2.	Problem Statement	4
3.	Project Objectives & Team Role Split	5
4.	System Architecture & Trust Boundaries	6
5.	Reconnaissance & Attack-Surface Inventory	8
6.	Literature / CVE Survey	10
7.	Implementation	12
8.	Vulnerability Demonstration (Exploitation)	14
9.	Remediation (Secure Redesign)	15
10.	Detection & Monitoring	16
11.	Testing & Results	17
12.	Conclusion	18
13.	References	19

1.0 INTRODUCTION

Web applications increasingly handle sensitive data — banking, payments, personal records — which makes access control critical. Authentication confirms who a user is; authorization determines what that user may access. A common and severe failure mode is Broken Object-Level Authorization (BOLA), typically demonstrated through Insecure Direct Object Reference (IDOR): an API exposes an object identifier (an account ID, a transaction ID) that the backend trusts without verifying it belongs to the requesting user.

If GET /api/accounts/101/transactions is not checked against the caller's identity, changing 101 to 102 can expose another user's data. This project builds a small banking web application — a login page, a dashboard, and a transaction view — and reproduces this exact failure on GET /api/accounts/{id}/transactions.

A vulnerable build intentionally omits the ownership check; a secure build restores it. Both run from the same codebase behind a single AUTHZ_MODE flag, which can be flipped live from the UI, so the entire difference between “vulnerable” and “fixed” is isolated to one authorization check rather than two diverging implementations. This makes the fix, and its effect, easy to observe directly. The purpose of this project is to understand the difference between authentication and authorization, and to show why sensitive APIs must never rely only on client-supplied identifiers.

2.0 PROBLEM STATEMENT

Scenario 4 — Broken Object-Level Authorization (BOLA) / IDOR Banking Demo

The banking application lets authenticated users view their transactions through GET /api/accounts/{id}/transactions. In the vulnerable build, the backend trusts the client-supplied account_id and never checks whether the requested account belongs to the caller. An authenticated user can therefore change the id in the request and read another user's transaction history.

Problem statement: How can a banking web application prevent an authenticated user from accessing another user's transaction records when the account identifier is supplied through a client-controlled API request?

The project answers this by first demonstrating the failure, then redesigning the backend so every account-scoped request is checked against the authenticated user's ownership before any data is returned.

3.0 PROJECT OBJECTIVES & TEAM ROLE SPLIT

Objectives

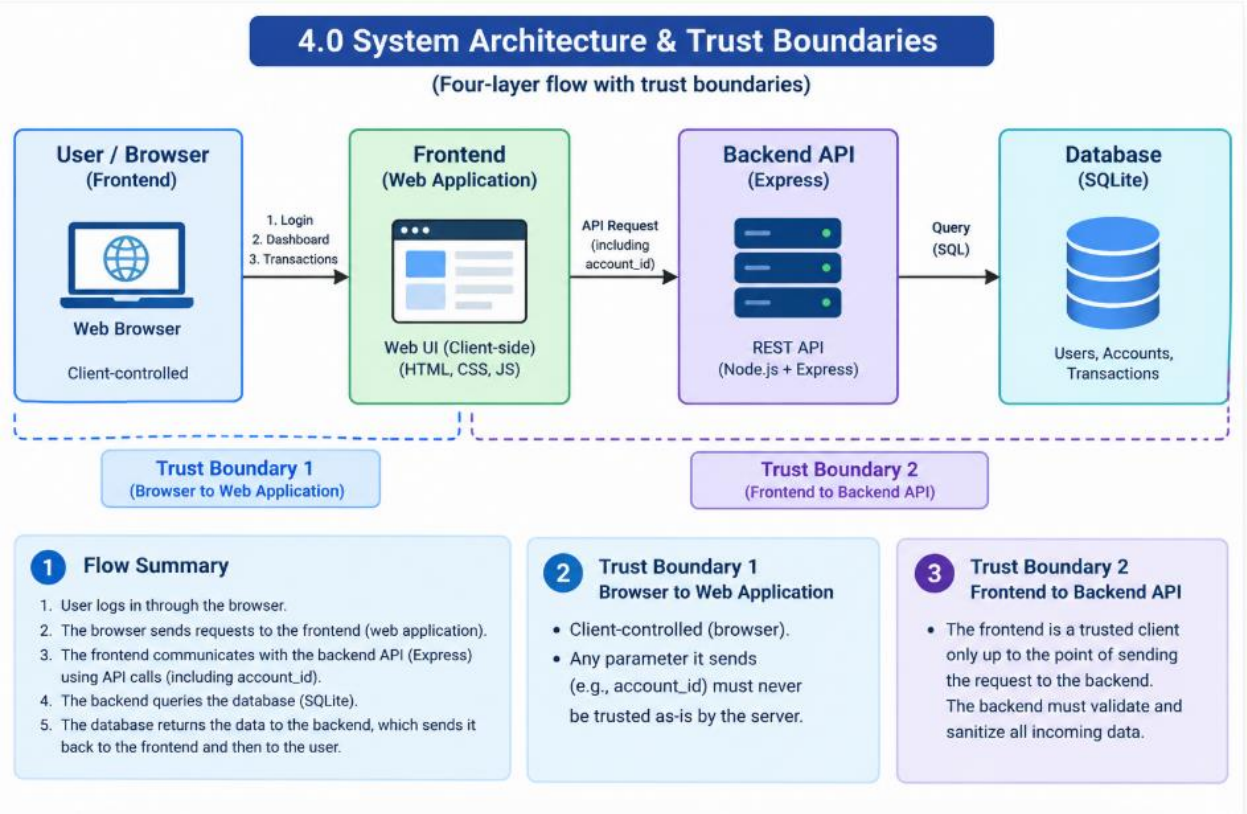
Build a working banking demo with a genuine login/authorization flow; reproduce a real BOLA/IDOR vulnerability rather than describe one abstractly; document the exact endpoint shape that makes it exploitable; implement a server-side fix; and re-test to prove the fix holds.

Team Role Split

Team Member	Role	Responsibility
Anshika	Backend Development	APIs, database, authentication logic, transaction retrieval
Aditya	Frontend Development	Login page, dashboard, transaction-view interface
Aryan	Exploit & Security Testing	Identified the vulnerable endpoint, tested account-ID manipulation
Aditya & Aryan	Defence Implementation	Server-side authorization checks, ownership validation, re-testing
Anshika	Report & Documentation	Architecture diagram, threat model, literature/CVE survey, this report

4.0 SYSTEM ARCHITECTURE & TRUST BOUNDARIES

The application follows a standard four-layer flow, matching the architecture designed at the start of the project:



On login, the frontend posts credentials to the backend, which verifies them against the database and returns a JWT. The frontend stores this token and attaches it as Authorization: Bearer <token> on every later request. The critical request in this project is GET /api/accounts/{id}/transactions.

Trust Boundary 1 — Browser to Web Application

The browser is client-controlled, so any parameter it sends — including account_id — must never be trusted as-is by the server.

Trust Boundary 2 — Frontend to Backend API

This is the boundary the vulnerability lives on. The frontend forwards a client-controlled account_id; the backend must verify the authenticated user is actually authorized for that specific account, not just that a valid session exists. Failure to perform this verification is the main cause of the vulnerability demonstrated in this project.

Trust Boundary 3 — Backend to Database

Only the backend talks to the database. Sensitive transaction data should only be queried after both authentication and authorization succeed; the database itself should never trust an account identifier received directly from the client.

Implementation

The original design specified MySQL; the working build uses Node's built-in node:sqlite module instead, to keep the project runnable with zero external services or native compilation. All other architectural elements — the layers, the JWT flow, and all three trust boundaries — match this design exactly.

5.0 RECONNAISSANCE & ATTACK-SURFACE INVENTORY

The attack surface of the application includes every page, API endpoint, and parameter through which a user can interact with the system. The most security-sensitive endpoint is GET /api/accounts/{id}/transactions, because the {id} value is supplied entirely by the client and directly identifies a bank account.

Method	Path	Client-controlled input	Auth Req.	Data Sensitivity
GET	/login, /signup	—	N	Low
POST	/api/auth/signup	username, password, full_name	N	Critical
POST	/api/auth/login	username, password	N	Critical
POST	/api/auth/logout	—	Y	Medium
GET	/api/me	—	Y	High
GET	/api/accounts	none (server-filtered)	Y	High
GET	/api/accounts/{id}	account_id	Y	High
GET	/api/accounts/{id}/transactions	account_id	Y	Critical
GET	/api/security-log	none (server-filtered)	Y	Medium

Primary Attack Surface

GET /api/accounts/{id}/transactions is the highest-risk endpoint in the system because it combines three properties at once: a client-controlled object identifier, authenticated-but-not-authorized access, and financial transaction data. By contrast, GET /api/accounts takes no identifier from the client at all — it is always filtered server-side to owner_id = <token's user> — so it is not part of the exploitable surface. The vulnerability is not “the API exposes account data”; it is specifically “the API trusts a client-supplied id instead of deriving it from the authenticated session.”

Data Sensitivity Classification

- Low: public information that does not contain sensitive user data.
- Medium: information related to sessions or basic application operations.
- High: personal account information or authenticated user details.
- Critical: credentials, banking records, transaction history, or other highly sensitive financial information.

6.0 LITERATURE / CVE SURVEY

Existing literature and publicly disclosed vulnerabilities on BOLA, IDOR, and authorization bypass were reviewed against trusted sources (OWASP, MITRE CWE, NIST NVD) to ground the demo in a real, recognized weakness class rather than an invented scenario.

#	Source	Date	Relevance
1	OWASP API1:2023 — Broken Object Level Authorization	5 Jun 2023	Defines BOLA exactly as implemented here
2	OWASP IDOR Prevention Cheat Sheet	Living doc.	Basis for the ownership-check fix in §9
3	CWE-639 — Authorization Bypass Through User-Controlled Key	Upd. 30 Apr 2026	Matches the account_id path parameter used here
4	CVE-2024-37889 — MyFinances	14 Jun 2024	Finance app exposed another user's financial data
5	CVE-2024-52294 — Khoj	30 Dec 2024	Authentication present, ownership authorization missing
6	CVE-2026-23844 — Whisper Money	19 Jan 2026	Personal-finance IDOR affecting other users' accounts
7	CVE-2026-53903 — MyComplianceOffice	1 Jul 2026	Authenticated user retrieved another user's document by ID

The consistent pattern across all seven sources — authentication present, authorization absent, financial or personal data exposed — is exactly what §8 and §11 reproduce and then close.

7.0 IMPLEMENTATION

Stack: Node.js + Express backend, SQLite via Node's built-in node:sqlite module (no native compilation required), JWT authentication (jsonwebtoken), password hashing (bcryptjs), and a plain HTML/CSS/JS frontend served statically by the same server.

Database Schema

Table	Key columns
users	id, username (unique), password_hash, full_name
accounts	id, owner_id → users, account_number, account_type, balance
transactions	id, account_id → accounts, type (credit/debit), amount, description, timestamp
access_logs	id, requester_id, requester_username, account_id, account_owner_id, endpoint, allowed, authz_mode, created_at

The Vulnerable / Secure Toggle

Rather than building two separate applications, both behaviours live in one function in src/routes/accounts.js:

```
function resolveAccess(accountId, userId) {
  const account = findAccountById(accountId);
  if (!account) return { account: null, exists: false, allowed: false };
  const allowed = config.authzMode === "secure"
    ? account.owner_id === userId   // ownership enforced
    : true;                         // vulnerable: any existing id is allowed
  return { account, exists: true, allowed };
}
```

A single AUTHZ_MODE setting (vulnerable | secure) decides which branch runs. This keeps the vulnerability isolated to one missing condition rather than a difference in overall design — a more honest demonstration of why BOLA happens: it is almost always one missing check, not a missing architecture.

Beyond the Base Specification

Three features were added to make the vulnerability and its fix observable rather than theoretical:

- Live mode toggle — POST /api/admin/authz-mode flips AUTHZ_MODE in memory instantly, so vulnerable and secure behaviour can be compared in one session without restarting the server.
- Request/response inspector — the transactions page shows the exact GET request, HTTP status, and JSON body for every account id tested, making the endpoint shape from §5 directly visible.

- Security Log (access_logs) — every call to the two account-scoped endpoints is recorded with requester, target account, and an ALLOWED/BLOCKED outcome, discussed further in §10.

8.0 VULNERABILITY DEMONSTRATION (EXPLOITATION)

With AUTHZ_MODE=vulnerable, the exploit was reproduced end-to-end:

1. Log in as alice (seeded demo user), whose own account is id 1.
2. On the transactions page, request a different id — 2, which belongs to bob — instead of 1.
3. The client sends GET /api/accounts/2/transactions with Alice's JWT in the Authorization header.
4. The server verifies the JWT (authentication passes) but, in vulnerable mode, never checks account.owner_id against the token's user id — so it returns account 2's data anyway.

Observed Response

Account 2 (Bob's) returned while authenticated as Alice:


```
{
  "account_id": 2,
  "transactions": [
    { "id": 5, "type": "credit", "amount": 1500,
      "description": "Salary deposit", "timestamp": "2026-09-22 14:08:52" },
    { "id": 6, "type": "debit", "amount": 350.6,
      "description": "Rent payment", "timestamp": "2026-09-22 14:08:52" }
  ]
}
```


This is Bob's data, obtained purely by changing an integer in the URL — no credentials, tokens, or sessions belonging to Bob were ever used. The same result is reproducible outside the browser with a single authenticated request:

```
curl -H "Authorization: Bearer <alice's JWT>"
http://localhost:4000/api/accounts/2/transactions
```

This confirms the exact failure mode from §2: the backend authenticates the request but never authorizes it against the specific resource requested.

9.0 REMEDIATION (SECURE REDESIGN)

The fix follows the OWASP IDOR Prevention Cheat Sheet's core guidance: never resolve an object by client-supplied id alone — always scope the lookup to the authenticated user.

```
function findOwnedAccount(id, ownerId) {
  return db.prepare(
    "SELECT * FROM accounts WHERE id = ? AND owner_id = ?"
  ).get(id, ownerId);
}
```

With AUTHZ_MODE=secure, resolveAccess() (§7) calls this instead of the unscoped findAccountById(). The owner_id = ? clause means a query for an account that exists but belongs to someone else returns no row at all — the database itself never returns the mismatched data. If the row is missing because of ownership rather than because the id doesn't exist, the endpoint responds 403 Forbidden: you do not own this account instead of leaking a 404-vs-403 distinction that would let an attacker enumerate valid ids.

This single change closes the vulnerability on both GET /api/accounts/{id} and GET /api/accounts/{id}/transactions, since both route handlers call the same resolveAccess() function — there is exactly one place in the codebase that decides account access, which is also the one place that needed fixing.

10.0 DETECTION & MONITORING

Blocking a bad request is not the same as knowing one happened. Every call to the two account-scoped endpoints — allowed or blocked — is written to access_logs with the requester, the target account's true owner, the endpoint, and the outcome. The Security Log page surfaces entries relevant to the viewer: requests they made, and requests anyone made against an account they own.

This matters specifically for BOLA/IDOR because the vulnerability produces no error, no crash, and no obviously anomalous traffic — a successful exploit looks identical to a normal authenticated request at the network level. Without logging tied to resource ownership (not just “who logged in”), a real vulnerable deployment would have no way to detect that Bob had ever accessed Alice's account.

Time	Requested by	Account ID	Result	Mode
...	bob	1 (Alice's)	ALLOWED	vulnerable
...	bob	1 (Alice's)	BLOCKED	secure

This is a defence-in-depth addition on top of the fix in §9, not a substitute for it — logging alone would not have stopped Bob from reading Alice's data in vulnerable mode, but it turns an otherwise silent breach into an auditable one, matching OWASP's broader logging-and-monitoring guidance for API security.

11.0 TESTING & RESULTS

The same test — Alice (authenticated) requesting Bob's account id — was run under both modes and compared directly:

Request	Vulnerable mode	Secure mode
GET /api/accounts/2	200, Bob's account returned	403, Forbidden: not owner
GET /api/accounts/2/transactions	200, Bob's transactions returned	403, blocked
Security Log entry	ALLOWED	BLOCKED
GET /api/accounts/1 (own account)	200, unaffected	200, unaffected

The fourth row is deliberate: the fix must not break legitimate access to a user's own data, only cross-account access. Requesting one's own account and requesting /api/accounts (the server-filtered list) behaved identically in both modes, confirming the change is scoped precisely to the ownership check and does not regress normal use. Re-testing after the fix used the exact same request — same user, same tampered id, same client — with only AUTHZ_MODE changed, which is what makes the before/after comparison a controlled one.

12.0 CONCLUSION

This project reproduced, exploited, and fixed a real class of vulnerability — Broken Object-Level Authorization — inside a working banking demo rather than describing it in the abstract. The central lesson is the one OWASP API1:2023 and CWE-639 both state directly: authentication and authorization are separate checks, and an API is only as safe as its weakest object-level check, not its login page.

The fix required exactly one additional SQL condition (owner_id = ?) applied consistently across every endpoint that accepts a client-controlled id — a small change with a disproportionate security impact, mirroring the real CVEs surveyed in §6 (MyFinances, Khoj, Whisper Money, MyComplianceOffice), all of which trace back to the same missing check. Extending the project with a live mode toggle, a request/response inspector, and an access log additionally demonstrated that a fix is only verifiable if it can be tested against its own prior failure under identical conditions, and that detection (logging by resource ownership) is a necessary complement to prevention, not an alternative to it.

13.0 REFERENCES

5. OWASP, “API1:2023 — Broken Object Level Authorization,” OWASP API Security Top 10, 5 Jun 2023.
6. OWASP, “Insecure Direct Object Reference Prevention Cheat Sheet,” OWASP Cheat Sheet Series (living document).
7. MITRE, “CWE-639: Authorization Bypass Through User-Controlled Key,” updated 30 Apr 2026.
8. NIST NVD, “CVE-2024-37889 — MyFinances,” 14 Jun 2024.
9. NIST NVD, “CVE-2024-52294 — Khoj,” 30 Dec 2024.
10. NIST NVD, “CVE-2026-23844 — Whisper Money,” 19 Jan 2026.
11. NIST NVD, “CVE-2026-53903 — MyComplianceOffice,” 1 Jul 2026.